NAME: AYAS AHAMED

ROLL NO: 2025115162

1. Write a C program to construct an expression tree for a given infix expression.

Example: (a-b)+(c*d)

Code:’

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

struct Node {
    char data;
    struct Node *left, *right;
};

struct Node *nodes[100];
char operators[100];
int ntop = -1, otop = -1;

struct Node* createNode(char data) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = data;
    n->left = n->right = NULL;
    return n;
}

int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    if (op == '^') return 3;
    return 0;
```

```
}

void makeTree() {
    char op = operators[otop--];

    struct Node *right = nodes[ntop--];
    struct Node *left = nodes[ntop--];

    struct Node *root = createNode(op);
    root->left = left;
    root->right = right;

    nodes[++ntop] = root;
}

struct Node* construct(char exp[]) {
    for (int i = 0; exp[i]; i++) {
        char ch = exp[i];

        if (isspace((unsigned char)ch))
            continue;

        if (isalnum((unsigned char)ch)) {
            nodes[++ntop] = createNode(ch);
        }
        else if (ch == '(') {
            operators[++otop] = ch;
        }
        else if (ch == ')') {
            while (otop >= 0 && operators[otop] != '(')
                makeTree();
```

```
            otop--;
        }
        else {
            while (otop >= 0 && operators[otop] != '(' &&
                (precedence(operators[otop]) > precedence(ch) ||
                (precedence(operators[otop]) == precedence(ch)
                 && ch != '^')))
                makeTree();

            operators[++otop] = ch;
        }
    }

    while (otop >= 0)
        makeTree();

    return nodes[ntop];
}

void inorder(struct Node *root) {
    if (root != NULL) {
        if (root->left) printf("(");

        inorder(root->left);
        printf("%c", root->data);
        inorder(root->right);

        if (root->left) printf(")");
    }
}

void preorder(struct Node *root) {
```

```
    if (root != NULL) {
        printf("%c", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

int main() {
    char exp[100];

    printf("Enter infix expression: ");
    scanf("%99s", exp);

    struct Node *root = construct(exp);

    printf("Inorder: ");
    inorder(root);

    printf("\nPreorder: ");
    preorder(root);

    return 0;
}
```

Sample Output

```
Enter infix expression: (a-b)+(c*d)

Inorder: ((a-b)+(c*d))
Preorder: +-ab*cd
```

2. Write a C program to check whether a binary tree satisfies the Binary Search Tree property.

Code:

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <limits.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* createNode(int data) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = data;
    n->left = n->right = NULL;
    return n;
}

int isBST(struct Node *root, long long min,
          long long max) {

    if (root == NULL)
        return 1;

    if (root->data <= min || root->data >= max)
        return 0;

    return isBST(root->left, min, root->data) &&
           isBST(root->right, root->data, max);
}

int main() {
    struct Node *root = createNode(50);

    root->left = createNode(30);
    root->right = createNode(70);
```

```
    root->left->left = createNode(20);
    root->left->right = createNode(40);

    root->right->left = createNode(60);
    root->right->right = createNode(80);

    if (isBST(root, LLONG_MIN, LLONG_MAX))
        printf("Given tree is a BST\n");
    else
        printf("Given tree is not a BST\n");

    return 0;
}
```

Sample Output

Given tree is a BST

3. Write a C program to implement a Max Heap with insertion and deletion operations.

Code:

```
#include <stdio.h>

#define MAX 100

int heap[MAX];
int size = 0;

void insert(int value) {
    if (size == MAX) {
        printf("Heap is full\n");
        return;
    }

    int i = size++;
    heap[i] = value;
```

```
    while (i > 0 && heap[(i - 1) / 2] < heap[i]) {
        int parent = (i - 1) / 2;

        int temp = heap[i];
        heap[i] = heap[parent];
        heap[parent] = temp;

        i = parent;
    }
}

void deleteMax() {
    if (size == 0) {
        printf("Heap is empty\n");
        return;
    }

    printf("Deleted: %d\n", heap[0]);

    heap[0] = heap[--size];

    int i = 0;

    while (1) {
        int largest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < size && heap[left] > heap[largest])
            largest = left;
```

```
        if (right < size && heap[right] > heap[largest])
            largest = right;

        if (largest == i)
            break;

        int temp = heap[i];
        heap[i] = heap[largest];
        heap[largest] = temp;

        i = largest;
    }
}

void display() {
    for (int i = 0; i < size; i++)
        printf("%d ", heap[i]);

    printf("\n");
}

int main() {
    int n, value;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    if (n < 0 || n > MAX) {
        printf("Invalid size\n");
        return 1;
    }
```

```
    printf("Enter elements: ");

    for (int i = 0; i < n; i++) {
        scanf("%d", &value);
        insert(value);
    }

    printf("Max Heap: ");
    display();

    deleteMax();

    printf("After deletion: ");
    display();

    return 0;
}
```

Sample Output

Enter number of elements: 5

Enter elements: 20 50 30 10 40

Max Heap: 50 40 30 10 20

Deleted: 50

After deletion: 40 20 30 10

4. Write a C program to check whether an undirected graph contains a cycle using DFS.

Code:

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int visited[MAX];
```

```
int n;

int dfs(int v, int parent) {
    visited[v] = 1;

    for (int i = 0; i < n; i++) {
        if (graph[v][i]) {

            if (!visited[i]) {
                if (dfs(i, v))
                    return 1;
            }
            else if (i != parent) {
                return 1;
            }
        }
    }

    return 0;
}

int hasCycle() {
    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            if (dfs(i, -1))
                return 1;
        }
    }

    return 0;
}
```

```
int main() {
    int edges, u, v;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    if (n < 1 || n > MAX)
        return 1;

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    printf("Enter edges:\n");

    for (int i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);

        if (u < 0 || u >= n || v < 0 || v >= n)
            return 1;

        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    if (hasCycle())
        printf("Cycle detected\n");
    else
        printf("No cycle detected\n");

    return 0;
}
```

Sample Input

Enter number of vertices: 4

Enter number of edges: 4

Enter edges:

0 1

1 2

2 0

2 3

Output

Cycle detected

5. Write a C program to count the number of islands in a grid containing L (land) and W (water).

Code:

```
#include <stdio.h>

#define MAX 100

char grid[MAX][MAX];
int visited[MAX][MAX];
int m, n;

int row[] = {-1,-1,-1,0,0,1,1,1};
int col[] = {-1,0,1,-1,1,-1,0,1};

void dfs(int x, int y) {
    visited[x][y] = 1;

    for (int k = 0; k < 8; k++) {
        int nx = x + row[k];
        int ny = y + col[k];

        if (nx >= 0 && nx < m &&
            ny >= 0 && ny < n &&
```

```
            grid[nx][ny] == 'L' &&
            !visited[nx][ny]) {

            dfs(nx, ny);
        }
    }
}

int countIslands() {
    int count = 0;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {

            if (grid[i][j] == 'L' &&
                !visited[i][j]) {

                dfs(i, j);
                count++;
            }
        }
    }

    return count;
}

int main() {
    printf("Enter rows and columns: ");
    scanf("%d %d", &m, &n);

    if (m < 1 || m > MAX || n < 1 || n > MAX)
        return 1;
```

```
    printf("Enter grid:\n");

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++)
            scanf(" %c", &grid[i][j]);
    }

    printf("Number of islands: %d\n", countIslands());

    return 0;
}
```

Sample Input

```
5 5
L L W W W
W L W W L
L W W L L
W W W W W
L L W L W
```

Output

```
Number of islands: 4
```

6. Write a C program to find the Minimum Spanning Tree using Kruskal's Algorithm.

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct Edge {
    int u, v, w;
};
```

```
struct Edge edges[MAX];
int parent[MAX];

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);

    return parent[x];
}

void unite(int a, int b) {
    parent[find(a)] = find(b);
}

int compare(const void *a, const void *b) {
    int x = ((const struct Edge*)a)->w;
    int y = ((const struct Edge*)b)->w;

    return (x > y) - (x < y);
}

int main() {
    int n, e, cost = 0, count = 0;

    printf("Enter vertices and edges: ");
    scanf("%d %d", &n, &e);

    if (n < 1 || n > MAX || e < 0 || e > MAX)
        return 1;

    printf("Enter u v weight:\n");
```

```
for (int i = 0; i < e; i++) {
    scanf("%d %d %d",
          &edges[i].u,
          &edges[i].v,
          &edges[i].w);

    if (edges[i].u < 0 || edges[i].u >= n ||
        edges[i].v < 0 || edges[i].v >= n)
        return 1;
}

for (int i = 0; i < n; i++)
    parent[i] = i;

qsort(edges, e, sizeof(struct Edge), compare);

printf("MST Edges:\n");

for (int i = 0; i < e; i++) {
    int u = edges[i].u;
    int v = edges[i].v;

    if (find(u) != find(v)) {
        unite(u, v);

        printf("%d - %d : %d\n",
               u, v, edges[i].w);

        cost += edges[i].w;
        count++;

        if (count == n - 1)
```

```
            break;
        }
    }

    if (count == n - 1)
        printf("Minimum Cost = %d\n", cost);
    else
        printf("Graph is disconnected; MST not possible\n");

    return 0;
}
```

Sample Input

Enter vertices and edges: 4 5

Enter u v weight:

0 1 10

0 2 6

0 3 5

1 3 15

2 3 4

Output

MST Edges:

2 - 3 : 4

0 - 3 : 5

0 - 1 : 10

Minimum Cost = 19

7. Write a C program to find the shortest distance from a source vertex to all other vertices.

Code:

```
#include <stdio.h>
#include <limits.h>
```

```
#define MAX 100
#define INF INT_MAX

int graph[MAX][MAX];
int dist[MAX], visited[MAX];
int n;

void dijkstra(int source) {
    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }

    dist[source] = 0;

    for (int count = 0; count < n; count++) {
        int u = -1;

        for (int i = 0; i < n; i++) {
            if (!visited[i] &&
                (u == -1 || dist[i] < dist[u]))
                u = i;
        }

        if (u == -1 || dist[u] == INF)
            break;

        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (graph[u][v] > 0 &&
                !visited[v] &&
```

```
                dist[u] <= INF - graph[u][v] &&
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("Vertex  Distance\n");

    for (int i = 0; i < n; i++) {
        if (dist[i] == INF)
            printf("%d       INF\n", i);
        else
            printf("%d       %d\n", i, dist[i]);
    }
}

int main() {
    int source;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    if (n < 1 || n > MAX)
        return 1;

    printf("Enter adjacency matrix:\n");

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &graph[i][j]);
```

```
            if (graph[i][j] < 0) {
                printf("Negative weights not allowed\n");
                return 1;
            }
        }
    }

    printf("Enter source vertex: ");
    scanf("%d", &source);

    if (source < 0 || source >= n)
        return 1;

    dijkstra(source);

    return 0;
}
```

Sample Input

```
Enter number of vertices: 4

Enter adjacency matrix:
0 4 1 0
0 0 2 5
0 1 0 8
0 0 0 0

Enter source vertex: 0
```

Output

```
Vertex  Distance
0       0
1       2
```

2 1

3 7

8. Write a C program to find a mother vertex in a directed graph.

Code:

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int visited[MAX];
int n;

void dfs(int v) {
    visited[v] = 1;

    for (int i = 0; i < n; i++) {
        if (graph[v][i] && !visited[i])
            dfs(i);
    }
}

int motherVertex() {
    int candidate = -1;

    for (int i = 0; i < n; i++) {
        if (!visited[i]) {
            dfs(i);
            candidate = i;
        }
    }

    for (int i = 0; i < n; i++)
```

```
        visited[i] = 0;

    dfs(candidate);

    for (int i = 0; i < n; i++) {
        if (!visited[i])
            return -1;
    }

    return candidate;
}

int main() {
    int e, u, v;

    printf("Enter vertices and edges: ");
    scanf("%d %d", &n, &e);

    if (n < 1 || n > MAX || e < 0)
        return 1;

    printf("Enter directed edges:\n");

    for (int i = 0; i < e; i++) {
        scanf("%d %d", &u, &v);

        if (u < 0 || u >= n || v < 0 || v >= n)
            return 1;

        graph[u][v] = 1;
    }
```

```
    int result = motherVertex();

    if (result == -1)
        printf("No mother vertex exists\n");
    else
        printf("Mother Vertex = %d\n", result);

    return 0;
}
```

Sample Input

Enter vertices and edges: 4 4

Enter directed edges:

0 1

1 2

2 3

0 3

Output

Mother Vertex = 0

9. Write a C program to find all Strongly Connected Components in a directed graph using **Kosaraju's**

Code:

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int transpose[MAX][MAX];
int visited[MAX];
int stack[MAX];
int top = -1;
int n;
```

```
void dfs1(int v) {
    visited[v] = 1;

    for (int i = 0; i < n; i++) {
        if (graph[v][i] && !visited[i])
            dfs1(i);
    }

    stack[++top] = v;
}

void dfs2(int v) {
    visited[v] = 1;
    printf("%d ", v);

    for (int i = 0; i < n; i++) {
        if (transpose[v][i] && !visited[i])
            dfs2(i);
    }
}

void kosaraju() {
    for (int i = 0; i < n; i++) {
        if (!visited[i])
            dfs1(i);
    }

    // Transpose graph
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            transpose[j][i] = graph[i][j];
    }
```

```
    for (int i = 0; i < n; i++)
        visited[i] = 0;

    printf("Strongly Connected Components:\n");

    while (top >= 0) {
        int v = stack[top--];

        if (!visited[v]) {
            dfs2(v);
            printf("\n");
        }
    }
}

int main() {
    int e, u, v;

    printf("Enter vertices and edges: ");
    scanf("%d %d", &n, &e);

    if (n < 1 || n > MAX || e < 0)
        return 1;

    printf("Enter directed edges:\n");

    for (int i = 0; i < e; i++) {
        scanf("%d %d", &u, &v);

        if (u < 0 || u >= n || v < 0 || v >= n)
            return 1;
```

```
        graph[u][v] = 1;
    }

    kosaraju();

    return 0;
}
```

Sample Input

Enter vertices and edges: 5 6

Enter directed edges:

0 1

1 2

2 0

1 3

3 4

4 3

Output

Strongly Connected Components:

0 2 1

3 4

10. Write a C program to determine whether a graph is bipartite using BFS and two-coloring.

Code:’

```
#include <stdio.h>

#define MAX 100

int graph[MAX][MAX];
int color[MAX];
int queue[MAX];
int n;
```

```
int isBipartite() {
    for (int i = 0; i < n; i++)
        color[i] = -1;

    for (int start = 0; start < n; start++) {

        if (color[start] != -1)
            continue;

        int front = 0, rear = 0;

        queue[rear++] = start;
        color[start] = 0;

        while (front < rear) {
            int u = queue[front++];

            for (int v = 0; v < n; v++) {
                if (graph[u][v]) {

                    if (color[v] == -1) {
                        color[v] = 1 - color[u];
                        queue[rear++] = v;
                    }
                    else if (color[v] == color[u]) {
                        return 0;
                    }
                }
            }
        }
    }
```

```
    return 1;
}

int main() {
    int e, u, v;

    printf("Enter vertices and edges: ");
    scanf("%d %d", &n, &e);

    if (n < 1 || n > MAX || e < 0)
        return 1;

    printf("Enter edges:\n");

    for (int i = 0; i < e; i++) {
        scanf("%d %d", &u, &v);

        if (u < 0 || u >= n || v < 0 || v >= n)
            return 1;

        graph[u][v] = 1;
        graph[v][u] = 1;
    }

    if (isBipartite())
        printf("Graph is Bipartite\n");
    else
        printf("Graph is not Bipartite\n");

    return 0;
}
```

Sample Input

Enter vertices and edges: 4 4

Enter edges:

0 1

1 2

2 3

3 0

Output

Graph is Bipartite